Data Structure Project I: Sort and Complexity

Shiqiang Wu 24300810019@m.fudan.edu.cn

I. Project Structure

- cpp_code: contains all C++ implementation of sort algorithms
  - benchmark.cpp
    code to generate random data to sort and output the result(operating time) to `result.csv`
  - **my_algorithm.h**
    implementation of `insertion sort`, `merge sort`, `quick sort`, `improved sort`
    including ***q1***, ***q2***, ***q3***, ***q5***
  - sort_test.cpp
    some test samples to justify the correctness of different sort algorithms
- scripts: python scripts to run the sort algorithms and visualize the results
  - run_benchmarks.py
    script used to run the `benchmark.cpp`
  - visualize_results.py
    generate the visualization of the operation time
  - regression.py
    visualize the nolinear regression, measure the constants of time complexities of `Quick Sort` and `Insertion Sort`
  - check_environment.py
    check the environment
- data: `csv` file to record the result and some graphs
  - results.csv
    operation time of different sort algorithm
  - qs_results.csv
    opertation time of **improved sorts** choosing different `k`
  - execution_time_boxplot.png
  - execution_time_plot.png
  - execution_time_regression.png

II. Dependence Packages and Operation Workflow

1. dependence packages:
   `pandas`, `matplotlib`, `seaborn`, `numpy`
2. operation workflow:
   - step0: check environment

     ```
     cd scripts
     python check_environment.py
     ```

   - step1: test sort algorithms correctness

     ```
     cd ../cpp_code
     g++ sort_test.cpp -o sort_test
     ./sort_test
     ```

   which should show this:

```
Insertion_sort Int: 0 1 2 2 3 4 5 5 7 8 9
Merge_sort Int: 0 1 2 2 3 4 5 5 7 8 9
Quick_sort Int: 0 1 2 2 3 4 5 5 7 8 9
Improved_sort Int: 0 1 2 2 3 4 5 5 7 8 9
Insertion_sort Float: 0.4 1.2 1.7 2 2.5 3 3.1 3.3 6.5 8.2 9.1
Merge_sort Float: 0.4 1.2 1.7 2 2.5 3 3.1 3.3 6.5 8.2 9.1
Quick_sort Float: 0.4 1.2 1.7 2 2.5 3 3.1 3.3 6.5 8.2 9.1
Improved_sort Float: 0.4 1.2 1.7 2 2.5 3 3.1 3.3 6.5 8.2 9.1
Insertion_sort Char: $ A F J K T U _ a b f s
Merge_sort Char: $ A F J K T U _ a b f s
Quick_sort Char: $ A F J K T U _ a b f s
Improved_sort Char: $ A F J K T U _ a b f s
```

- step2: run the test and visualize the operation time

```
cd ../scripts
python run_benchmarks.py
```

III. Operation Time of Different Algorithms and Reflection

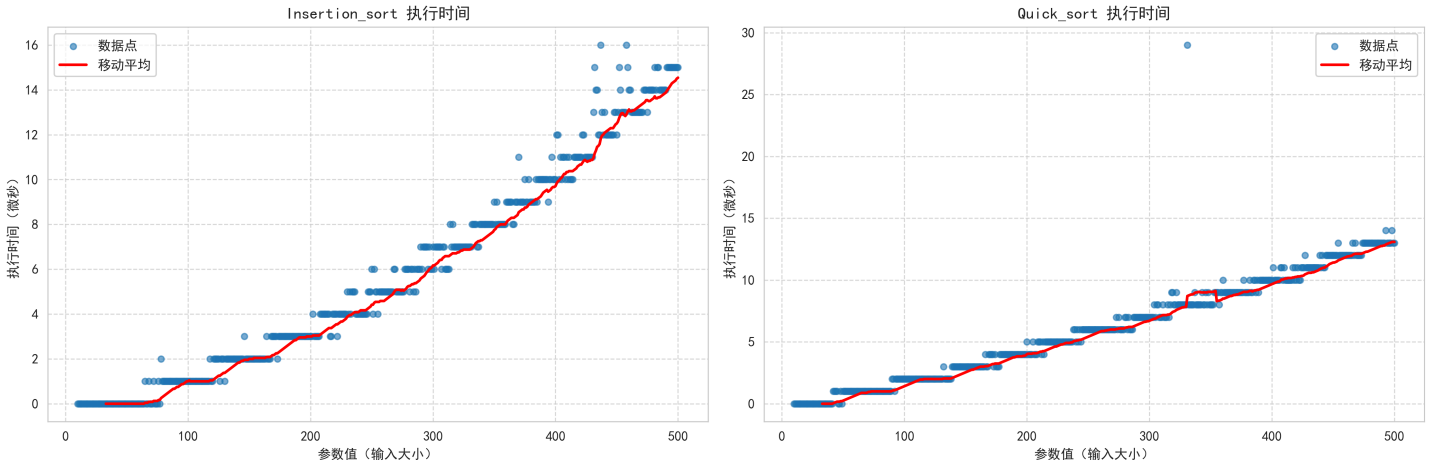


figure1: execution time plot(10-500)

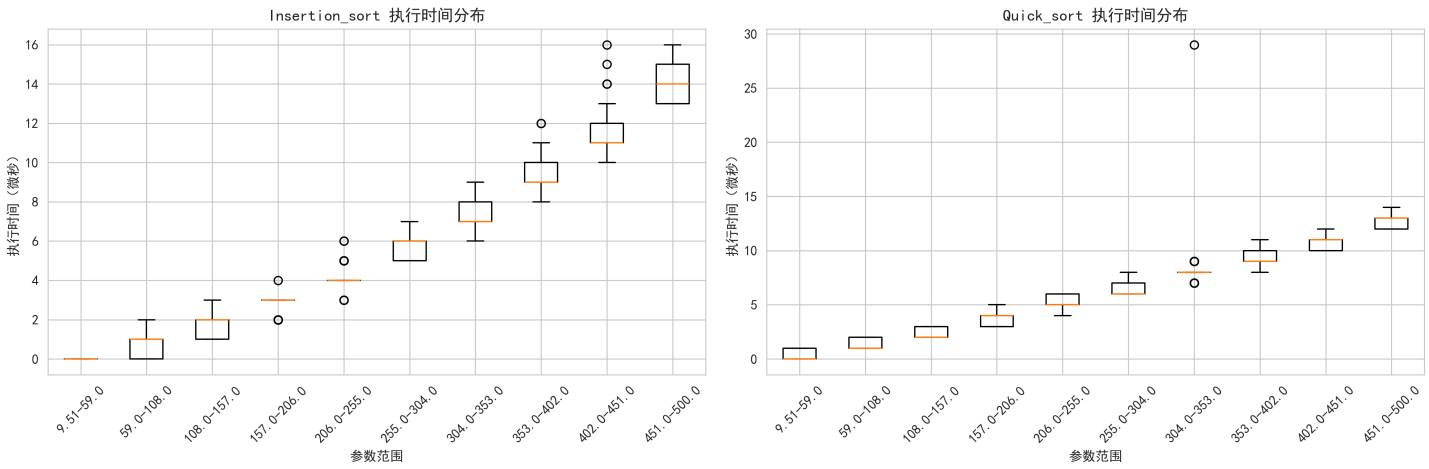


figure2: time bloxplot(10-500)

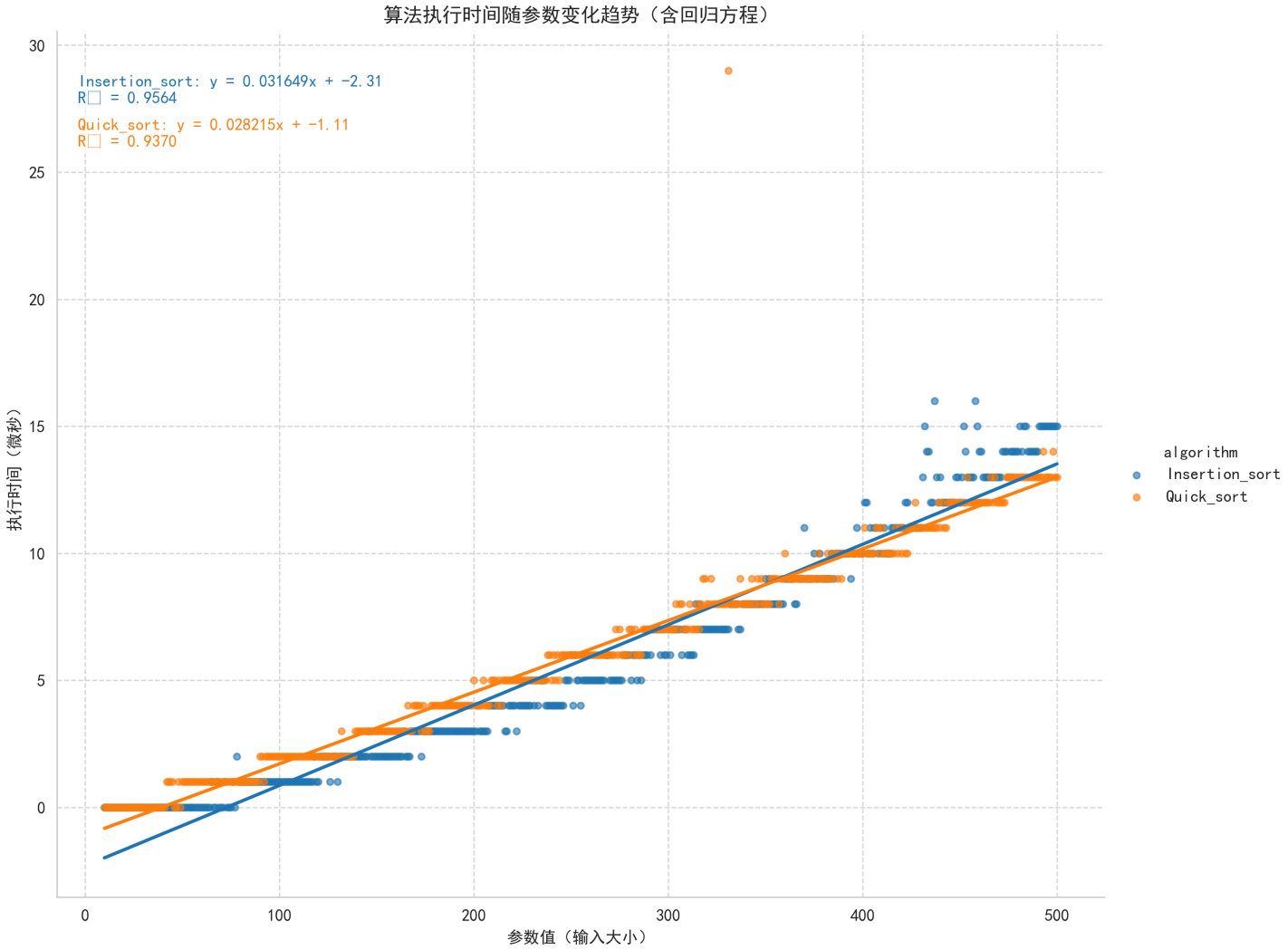


figure3: time regression plot and comparison of different algorithm(10-500)

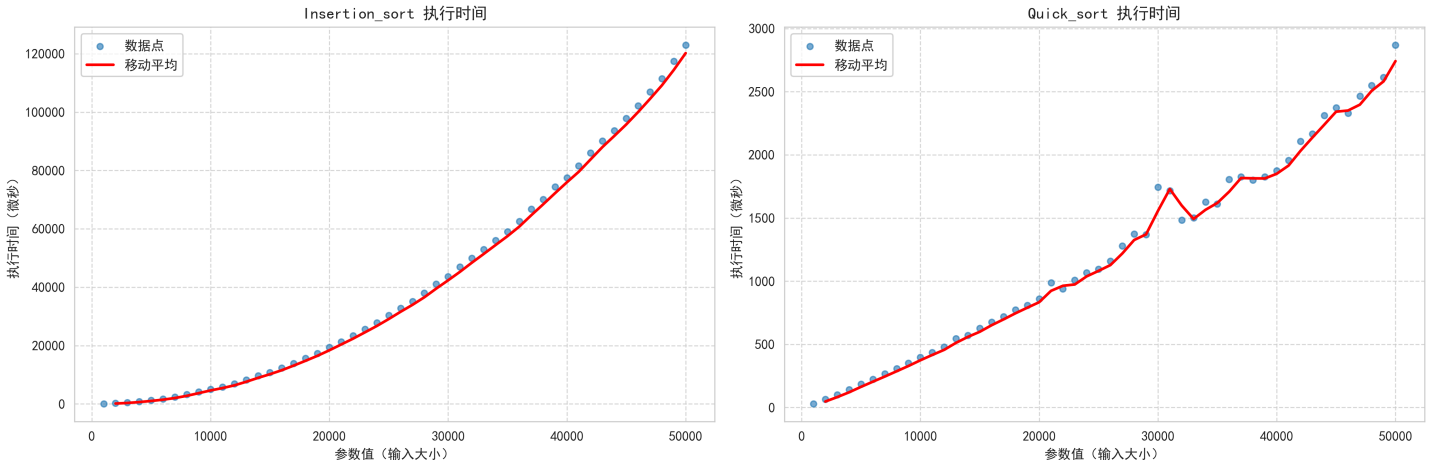


figure4: execution time plot(1000-50000)

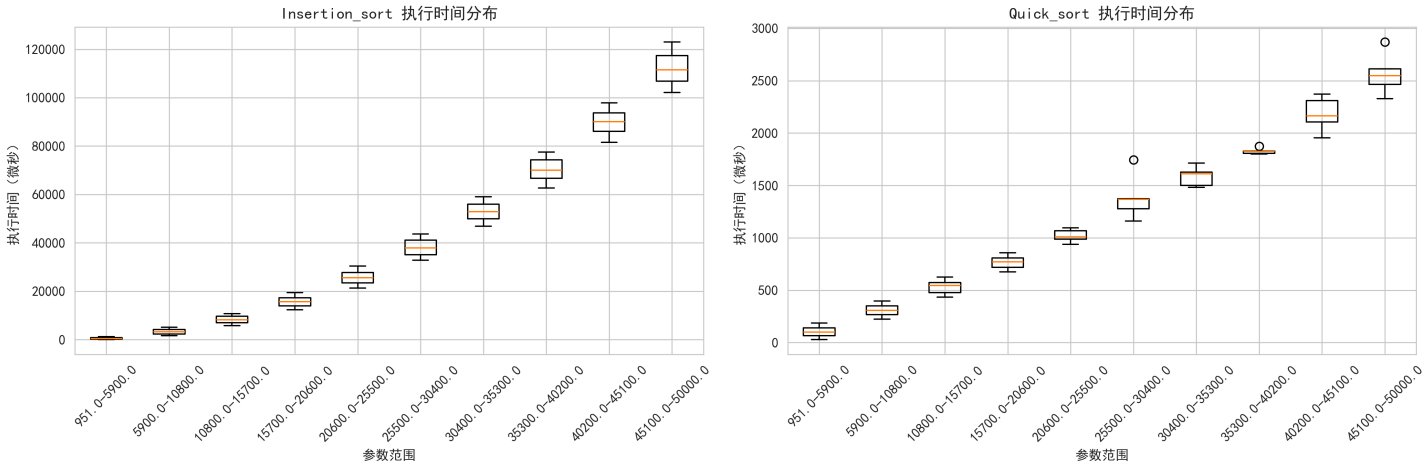


figure5: time bloxplot(1000-50000)

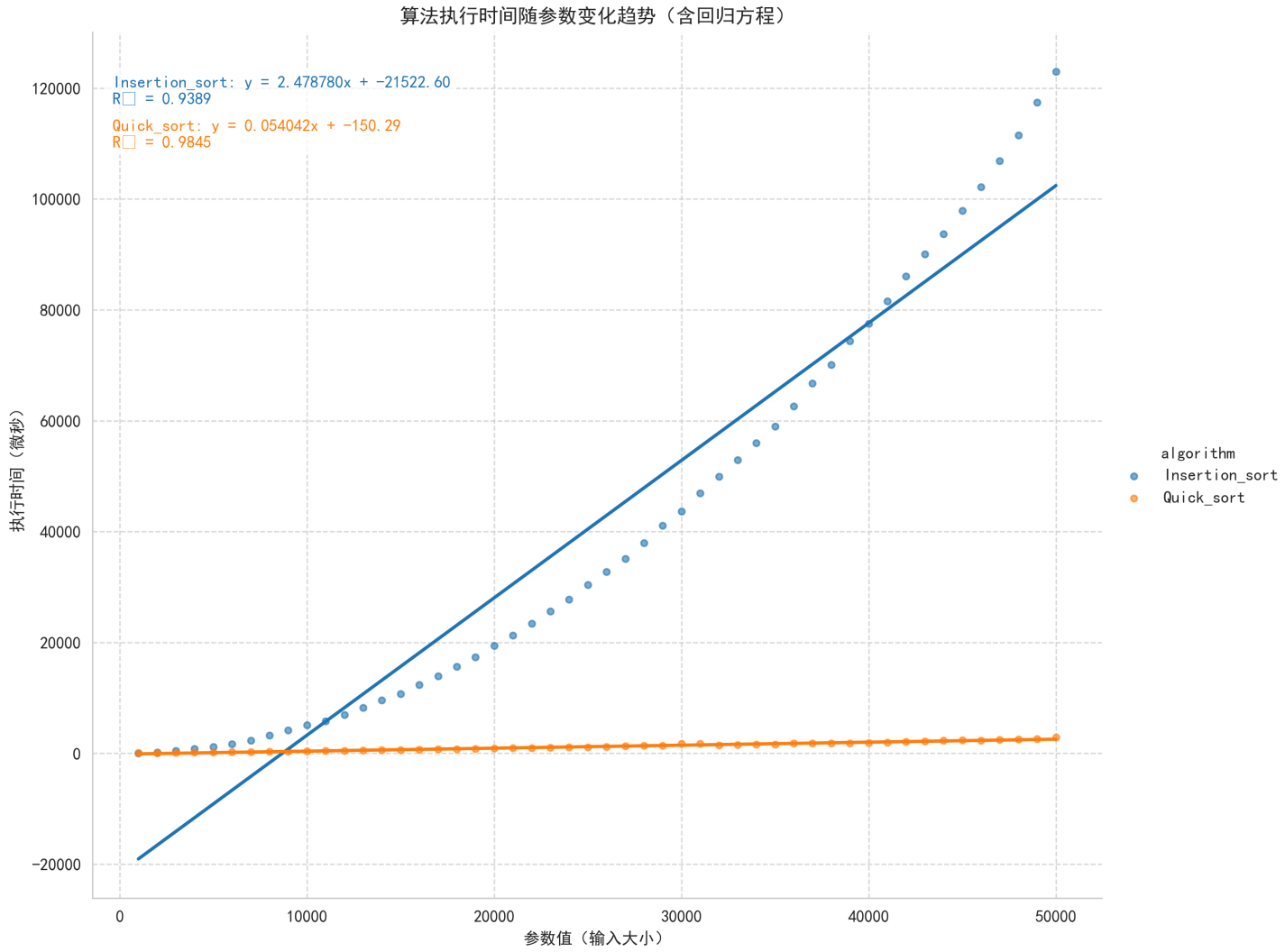


figure6: time regression plot and comparison of different algorithm(1000-50000)

1. **Compare the performance of Quick sort and Insertion sort(with tiny scale data) and linear regression**

   In figure 1-3, we compare algorithms with tiny scale data(size range from 10 to 500, step=1), for each size we repeat the experiment with 10 times and get the average result to reduce the randomness error.Notice that in figure3, we use the linear regression to fit the curves of different algorithms.

   Here is the linear fit functions:

   Insertion sort:

$$y = 0.031649x - 2.31$$

Quick sort:

$$y = 0.028215x - 1.11$$

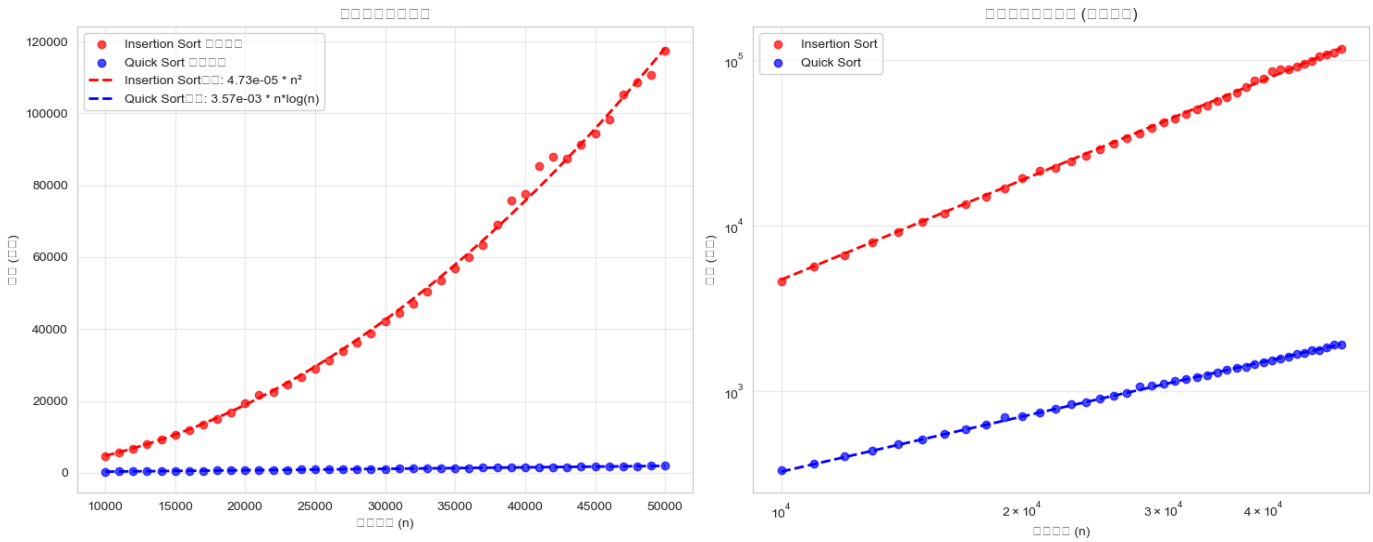
We then can compute the point where Quick sort outperforms Insertion sort:

$$x = 349.45$$

2. **Analysis of the Constants of Complexities of Quick Sort and Insertion Sort, Theorical Prediction for `improved_sort`**

```
============================================================
回归分析结果
============================================================
Insertion Sort (O(n²)) 回归常数: 4.726922e-05
Quick Sort (O(n log n)) 回归常数: 3.570257e-03

拟合质量评估 (R²值):
Insertion Sort R²: 0.997893
Quick Sort R²: 0.999143
```



We know that

$$c_i = 4.727 \times 10^{-5}$$

$$c_q = 3.570 \times 10^{-3}$$

After getting the constants, we can now compute the time complexity exactly.

- For `Quick Sort`:

$$T(N) = c_q N \ln(N)$$

- For `Insertion Sort`:

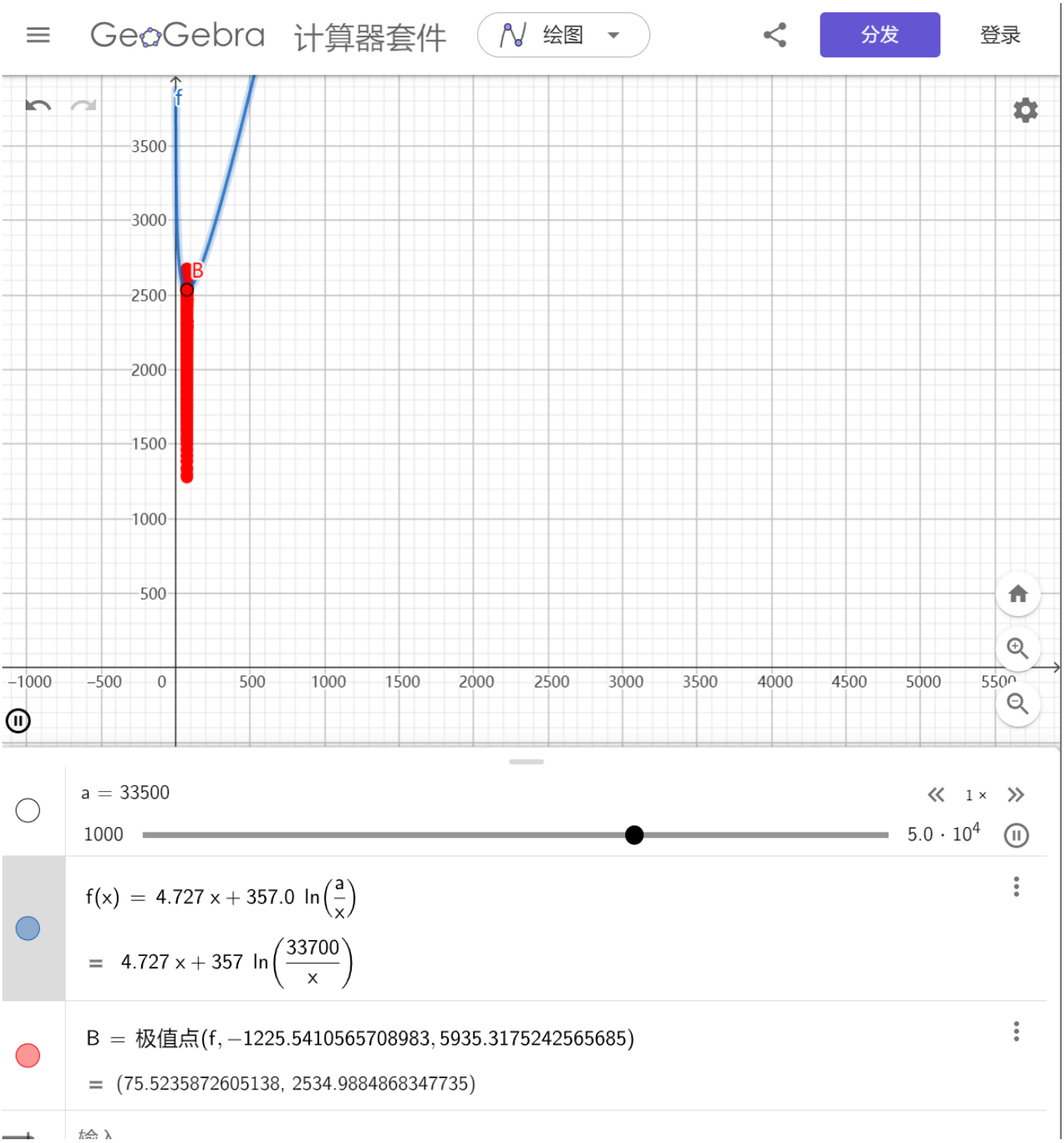
$$T(N) = c_i N^2$$

- For `Improved Sort`:
  We need $\frac{N}{k}$ times of Insertion sort, each with a subarray of length k.We need `Quick Sort` down to subarrays down to size of k, so that the recursive tree's height is $log(\frac{N}{k})$.Thus:

$$T(N) = c_i N k + c_q N \ln(\frac{N}{k})$$

$$T(N) = 4.727 \times 10^{-5} N k + 3.570 \times 10^{-3} N \ln(\frac{N}{k})$$

We can use the `Geogebra` to analyze the function(divided by `N`), `N` ranges from `1000` to `50000`



We can find that when `N` (a in the picture) changes, the minimum doesn't change much.The best `k` theorically thus is about: `k=75`

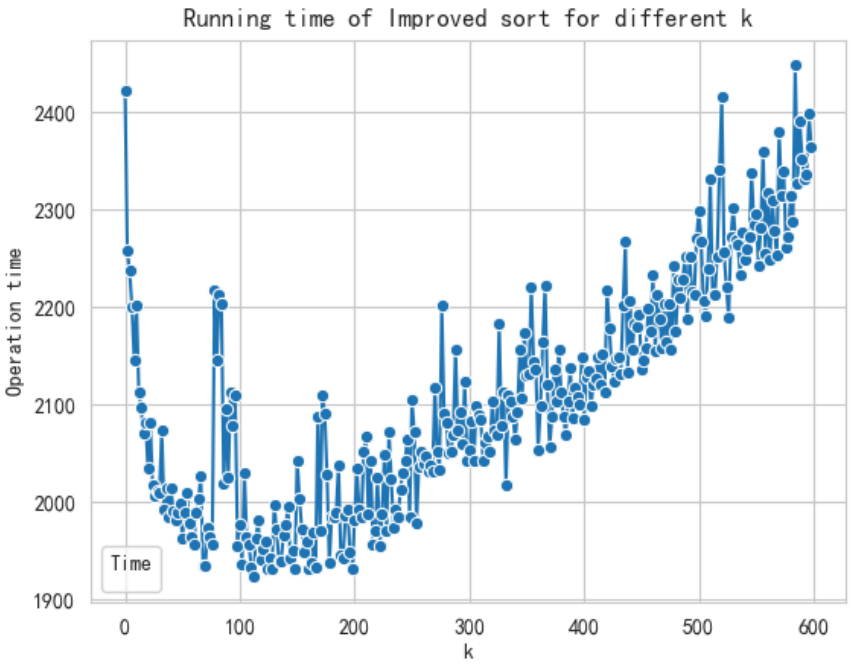
IV. Improved Sort k selection

Our implementation of `Improved Sort` is simply a combination of quick sort and insertion sort.

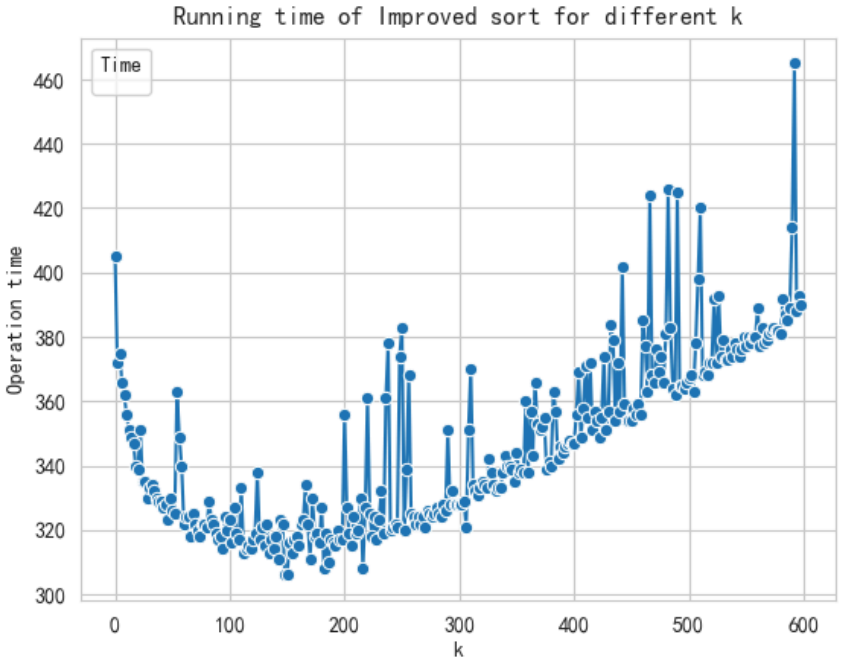
```
template<typename T>
void Improved_Sort_k(std::vector<T>& arr, int k){
// To-Do: Implement the algorithm combining the quick sort and insertion sort
   Quick_sort_naive_k(arr, k);
   Insertion_sort(arr);
}
```

The key problem is the choice of k.
We can try different `k` s, for each we experiment for 20 times and get the average time.The visualization of the

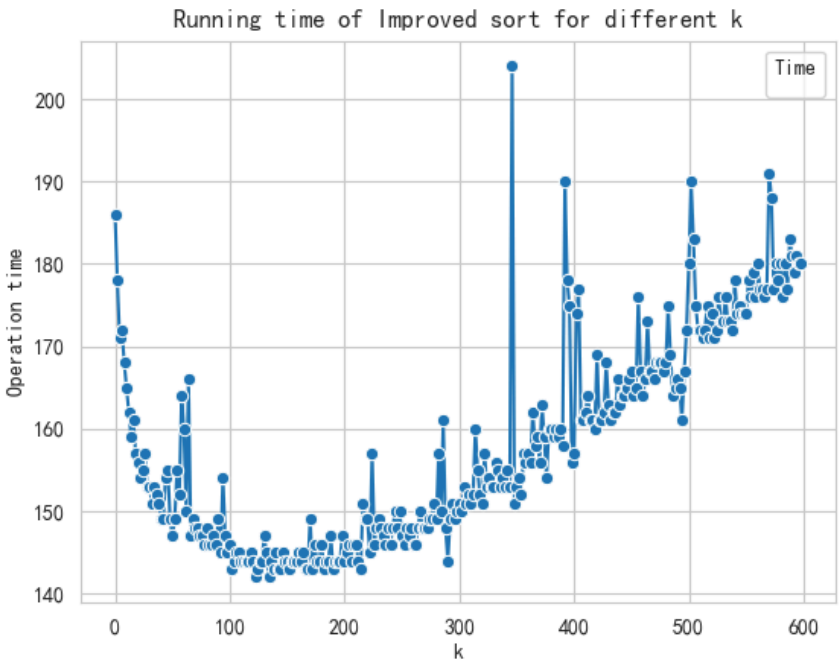
results(`qs_results.csv`) is as follows.



array_length = 50000



array_length = 10000



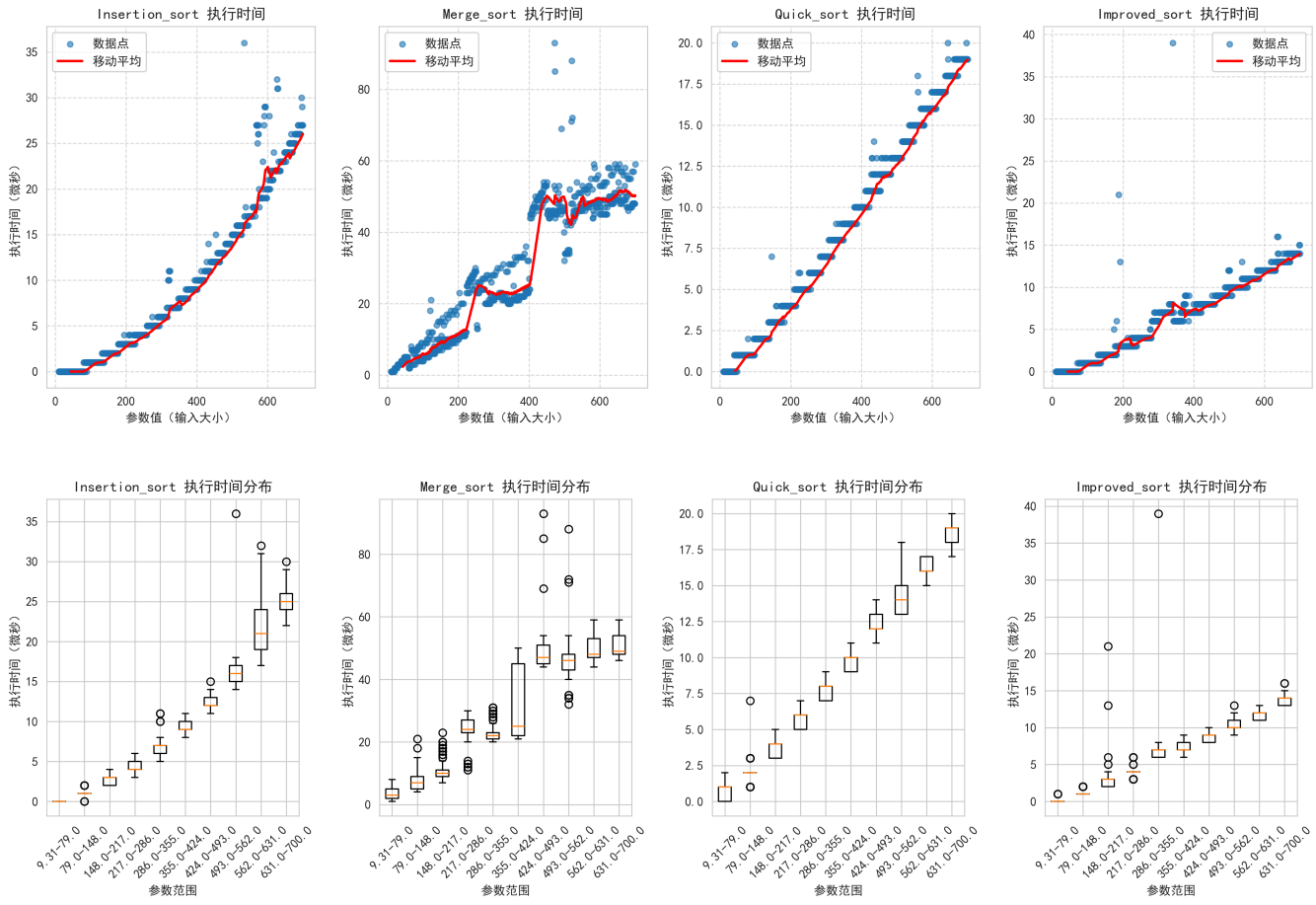
array_length = 5000

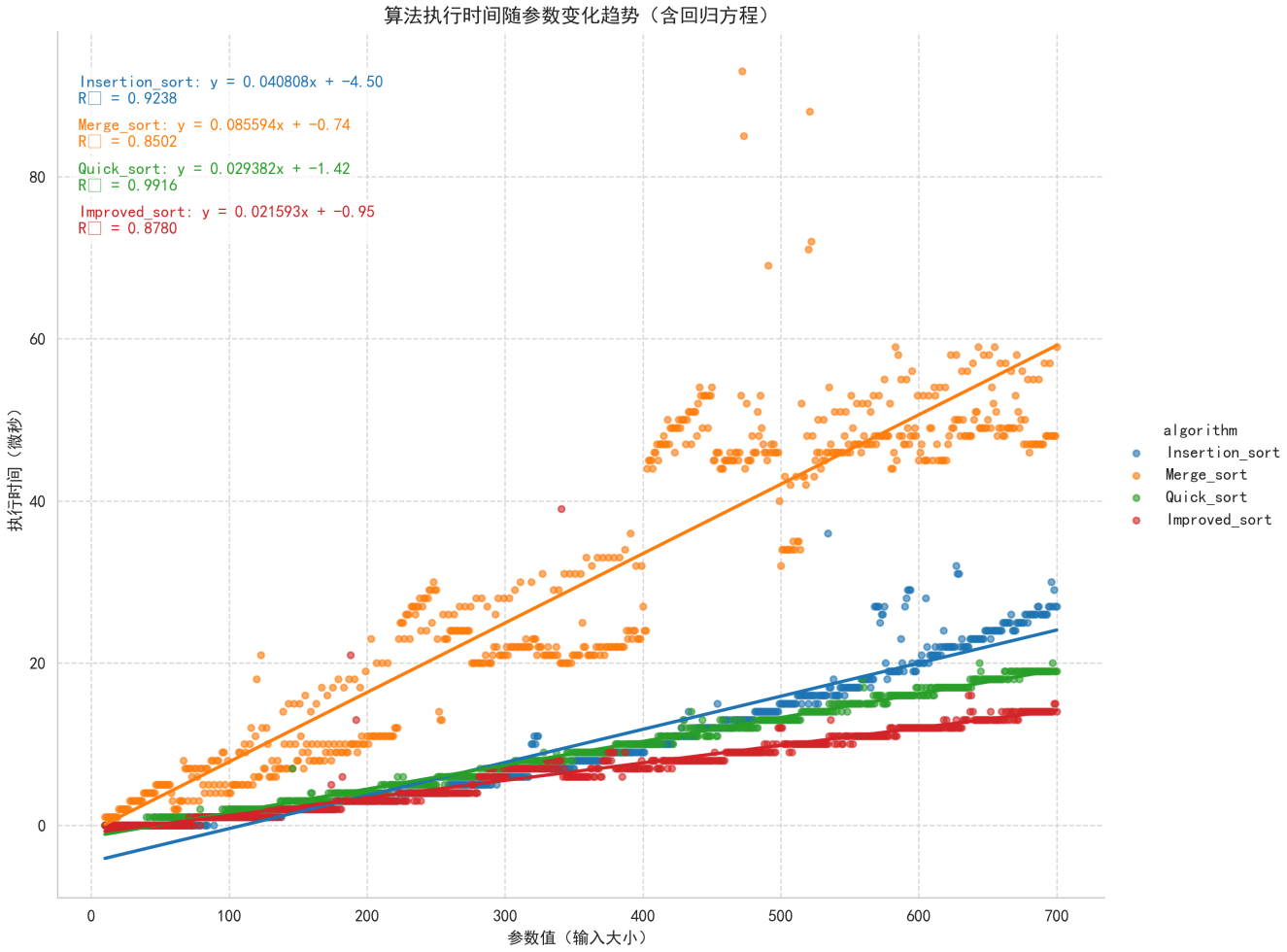
We can find that when `k=120` or so the operation time is the shortest.This is different from the therotical calculation we made above of about `75`.
We then can set `k=120` to compare our improved sort with other algorithms

V.How about Merge Sort?

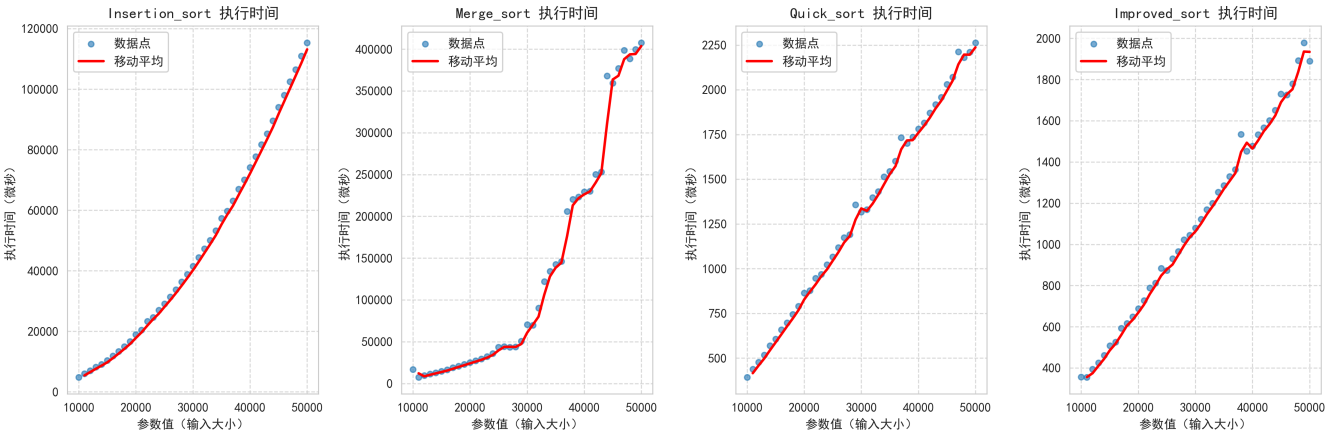
We haven't talk about the `Merge sort` yet, since in the experiment we find that in practice the `Merge sort` can't beat the `Insertion sort`, not to say `Quick_sort` and `Improved_sort` in the scale of data my laptop allows.

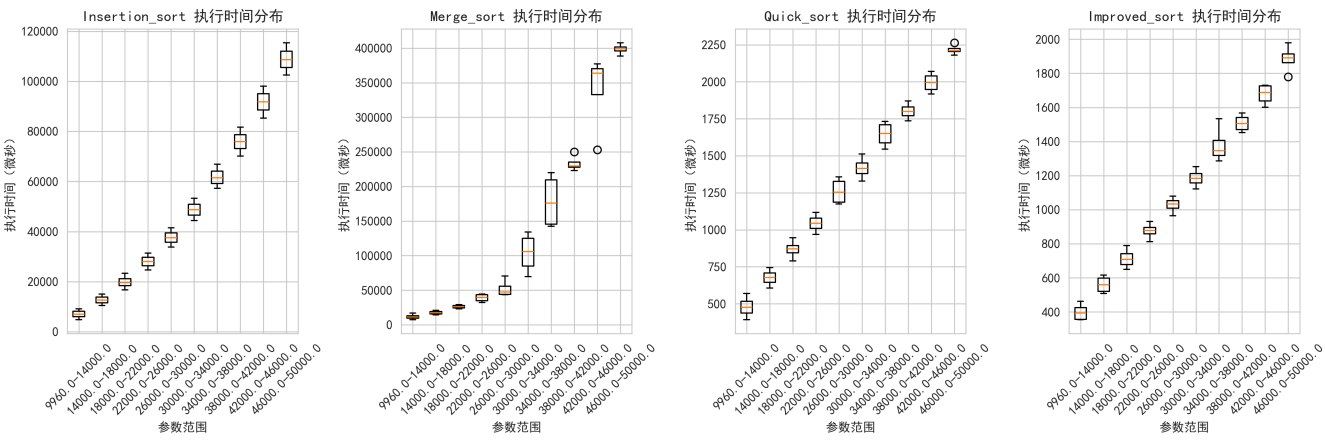
- **(10-700)**

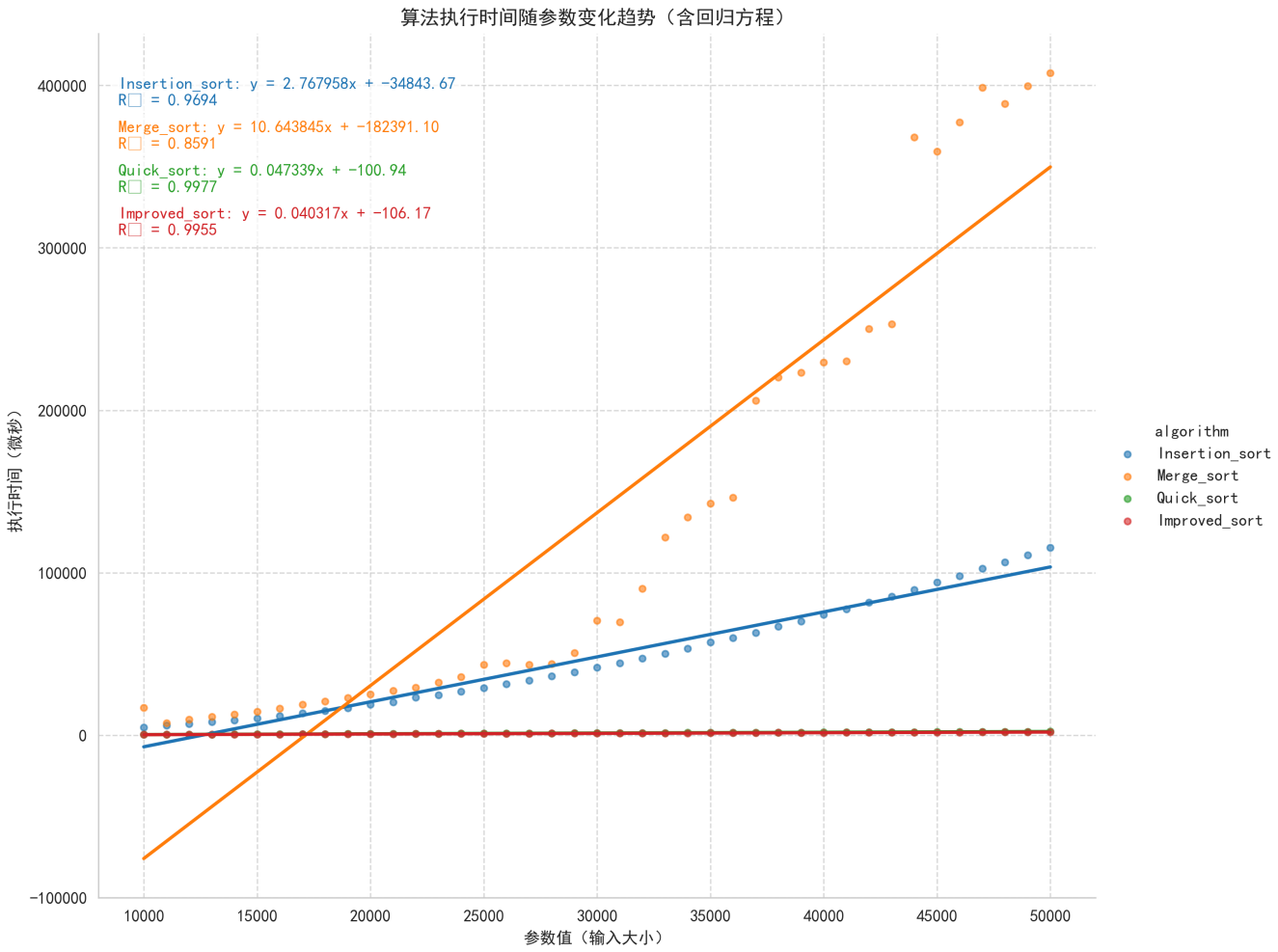



算法执行时间随参数变化趋势（含回归方程）
Insertion_sort: y = 0.040808x + -4.50
R□ = 0.9238
Merge_sort: y = 0.085594x + -0.74
R□ = 0.8502
Quick_sort: y = 0.029382x + -1.42
R□ = 0.9916
Improved_sort: y = 0.021593x + -0.95
R□ = 0.8780
执行时间（微秒）
参数值（输入大小）
algorithm
Insertion_sort
Merge_sort
Quick_sort
Improved_sort

- **(10000-50000)**


Insertion_sort 执行时间
Merge_sort 执行时间
Quick_sort 执行时间
Improved_sort 执行时间
数据点
移动平均
执行时间（微秒）
参数值（输入大小）


Insertion_sort 执行时间分布
Merge_sort 执行时间分布
Quick_sort 执行时间分布
Improved_sort 执行时间分布
执行时间（微秒）
参数范围



The main reason is that Merge_sort_1 and Merge_sort_2 functions have a significant performance overhead due to frequent memory allocation and copying.

The Problem with Memory Allocation
Creating a new vector on the heap (std::vector) is a costly operation. It involves:

- Requesting memory from the operating system, which is a time-consuming task.
- Copying the data from one location to another.

In a highly recursive algorithm like merge sort, this overhead adds up quickly. While the time complexity of your algorithm remains $O(NlogN)$ from a theoretical standpoint, the constant factor is very large due to these inefficient operations.

In contrast, other algorithms in your code, like Insertion_sort and Quick_sort_naive, operate in-place or with minimal extra space.

VI.Artificial Intelligence Usage Clarification

Some of the visualization part scripts codes(`run_benchmarks.py` , `visualize_results.py`) are written by `ChatGpt` and `Gemini` , with revision of my own.All the implementation of sort algorithms is written by me manualy.All experiments were conducted on my own.